Caruca v2

LLM-based specification mining for opaque shell commands

Component & Functionality Spec

Inputs, outputs, and the CLI surface per component

SOURCE	ai_docs/prep/component_functionality.md
GENERATED	August 29, 2026 at 7:08 PM
PAGES	9

Table of contents

Component & Functionality Spec

Components Covered

Five priority components, built in this order per CLAUDE.md's stated build order (baseline + evaluation harness first, then naive-LLM baseline, before sandbox/agentic work):

1. Baseline instrumentation (extends v1)
2. Naive-LLM baseline (new)
3. Secure sandbox environment (new, extended scope)
4. LLM tool-augmentation layer (new)
5. Evaluation harness (extended scope)

Plus one deferred/optional component, design captured but not prioritized:

6. Web GUI (optional, stretch)

Build Tiering

Per the review in `~/.claude/plans/review-our-progress-so-delightful-token.md` (2026-08-29): the 5 priority components stage into two tiers, not one flat MVP. **Tier 0** (Baseline Instrumentation + Naive-LLM Baseline, plain docs only + Evaluation Harness's Q2-style command-level comparison + telemetry

- variance sampling) is the actual minimal deliverable answering Eiers' "does a naive prompt work, and at what cost" mandate — no sandbox required. **Tier 1** (Secure Sandbox both profiles, LLM Tool-Augmentation, real execution-based Q1, Web GUI) is the gap-driven extension path. Full sequencing/roadmap detail belongs in the `10_generate_build_order_worker.md` pass, not here — this note just flags that I/O mappings below aren't meant to be built in one flat pass.

Per-Component Input/Output Mapping

Baseline instrumentation (extends v1)

- Input: a command name → v1's existing `caruca syntax-spec CMD` invocation
- Processing: wraps the existing DSPy/LLM call unmodified, capturing tokens/cost/wall-clock/model-ID/seed around it. Requires the DSPy 3.3.1 migration fix noted in CLAUDE.md as a prerequisite, not optional.
- Output: v1's existing syntax-spec Python file (unchanged format) + a telemetry record (JSON) alongside it. Feeds both the Evaluation Harness (Q2 comparison input) and Secure Sandbox (config-gen input, same as v1's pipeline today).

Naive-LLM baseline (new)

- Input: a command's binary name + its documentation (man page text or `--help` output). Optionally, augmented documentation context from the Tool-Augmentation layer, but only via an explicit separate flag (see below) — never silently merged with the plain-docs run.
- Processing: one straightforward, few-shot-primed prompt; no agentic tool use, no validation/retry loop (deliberately, per Eiers' "don't min-max the perfect specification synthesizer" guidance).
- Output: a specification in the **same Python-DSL shape v1 uses**. This is the key cross-component design decision: because the naive-LLM's spec occupies the exact same pipeline position as v1's own LLM-generated syntax spec, it can flow through the same downstream config-gen → Secure Sandbox trace → annotate pipeline that v1 already runs — giving two comparisons from one design choice:
  - Q2-style (syntax-spec accuracy): naive-LLM's spec vs. v1's spec vs. ground-truth spec, via `cmp_specs.py`, no tracing needed
  - Annotation-diff comparison (a Q1-*adjacent* check, not the paper's actual Q1 methodology — see Open Questions): naive-LLM's spec, run through the same config-gen/trace/annotate stages as v1's, producing a final annotation diffable against `benchmarks/annotations/`. The paper's real Q1 numbers (PaSh 52/52, ShellCheck 6/6, Shseer 18/18) are execution-verified — real test suites rerun with Caruca's output plugged in, not diffed — which this does not attempt; that's a distinct, heavier Tier 1 task Plus a telemetry record (tokens/cost/wall-clock/model-ID/seed).
- Naive-LLM (augmented docs) is tracked as a **distinct, separately labeled condition** from Naive-LLM (plain docs) throughout — never blended into one baseline number, per the explicit scoping decision that the naive baseline itself should stay man-page-only.

Secure sandbox environment (new, extended scope)

- Input: a command's syntax spec (Syntax IR, from either v1 or the naive-LLM baseline) + an environment **profile** selector + an isolation-backend selector.
- Processing, two sub-flows:
  - Fixture/environment generation, selectable by profile:
    - `v1-faithful`: exact replication of v1's existing fixture set (no symlinks/permissions/pipes/etc.) — the apples-to-apples control
    - `v2-extended`: the broadened `EnvironmentArgument` model — symlinks (file/dir/dangling), permission states, 2-3 level directory trees, correlated multi-file content (identical/disjoint/subset pairs), pipe/SIGPIPE cases, empty content, wider integer range (typical + large, not just -1/0/1), and environment-variable variation (paper §2.2's own disclosed limitation, gap #14 in `evaluation_gaps.md` — Caruca generates no env vars at all, so command behavior that branches on them is untested by either v1 or v2's v1-faithful profile)
  - Isolated execution: runs invocation permutations inside the chosen backend (Docker/overlayfs extension, Firecracker, or gVisor — backend choice still open), preserving strace/ptrace visibility, tearing down automatically on completion or crash.
- Output: a `Traces` JSON compatible with v1's existing schema (extended with fields for the new fixture types) + execution telemetry (wall-clock per invocation, isolation backend used, teardown status). Both profiles' Traces flow through v1's existing annotate step (unmodified) to the Evaluation Harness, **tagged by profile** rather than blended — this is what makes the v2-extended coverage-extension claim measurable on its own terms rather than averaged away.

LLM tool-augmentation layer (new)

- Input: a command binary whose documentation is sparse/absent, or an explicit augmentation request.
- Processing: starter toolset (binary/format inspection, static/code analysis, help/usage-string extraction) the LLM can call before producing a spec.
- Output: an augmented documentation-context file + per-tool-call telemetry. Consumed only by the Naive-LLM baseline, only via an explicit flag, and only as a separately labeled condition (never merged into the plain-docs baseline numbers).

Evaluation harness (extended scope)

- Input: v1's output, the naive-LLM baseline's output (plain and augmented-docs conditions, separately), and ground truth (`benchmarks/annotations/`).
- Processing: reuses `cmp_specs.py` methodology for Q2-style comparisons; adds:
  - Repeated-sampling variance runs (N independent generations per command, for both v1's LLM step and the naive-LLM baseline) — turns the single-run correctness number into one with reportable variance (dimension 4, "LLM nondeterminism is itself a result")
  - Disaggregated coverage computation (a strong-normalization-only subtotal alongside the blended coverage %), by default, not behind a flag
  - A three-condition comparison per command: v1-faithful (control), v2-extended (the actual coverage-extension claim), and the naive-LLM's spec run through both profiles
- Output: three-way comparison (v1 vs. naive-LLM vs. ground truth) + percent-change roll-up per dimension
  - a variance report + a disaggregated coverage report, all in a form usable directly in a paper (tables/figures/methodology description).

Web GUI (optional, stretch — design captured, build deferred)

- Input: a command name (or a saved run), triggering both v1's and v2's CLI invocations.
- Processing: backend spawns v1's CLI and `caruca-v2` 's subcommands each inside a PTY (preserving terminal semantics: colors, cursor control, progress output), streams both over websocket to the browser; reads the same telemetry JSON files each subcommand already writes rather than computing anything new.
- Output: two synchronized xterm.js-rendered terminal panes (left = v1, right = v2) in-browser, plus a metrics panel sourced from existing telemetry records.
- Purely a presentation layer over the other five components' existing CLI surfaces and telemetry outputs — costs those components nothing to add later, since it spawns their existing subcommands as subprocesses rather than requiring a new data format.

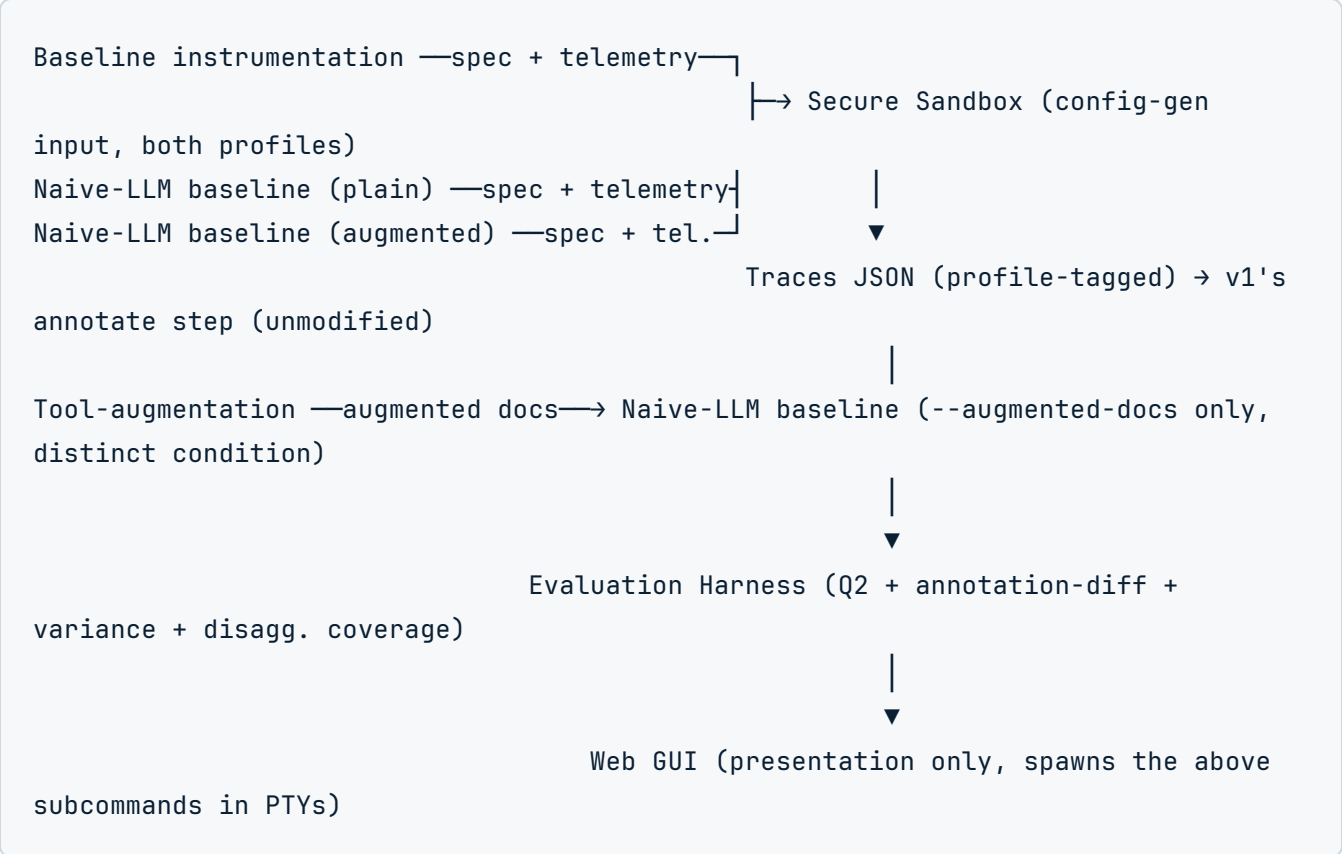
CLI Surface

One entry point, `caruca-v2` , subcommand-per-stage, mirroring v1's own `caruca <subcommand> COMMAND [flags]` shape. Integration with v1 is via **subprocess-wrapping**, not a library import — v1 requires Python ≥3.12 while this dev machine's system Python is 3.11, the trace phase must run on a different (provisioned) host than the dev machine, and v1 already exposes a stable file-based interface (the `Traces` JSON) that's more robust to depend on than parsing v1's CLI stdout.

```
caruca-v2 baseline syntax-spec CMD [--model MODEL] [--seed N] [--out DIR]

caruca-v2 naive-llm CMD [--docs PATH] [--augmented-docs PATH] [--model MODEL] [--
seed N] [--out DIR]

caruca-v2 sandbox generate CMD --profile v1-faithful|v2-extended [--number]
caruca-v2 sandbox trace CMD --profile v1-faithful|v2-extended [--backend
docker|firecracker|gvisor] [--length-only]

caruca-v2 augment CMD [--tools binary-inspect,static-analysis,help-extract] [--out
PATH]

caruca-v2 compare CMD [--dimension all|correctness|cost|coverage|consistency] [--
variance N] [--profile v1-faithful|v2-extended|both] [--out DIR]

caruca-v2 web serve [--port N]
```

Every telemetry-producing subcommand accepts `--out DIR`, defaulting to a project-standard location (e.g. `eval/runs/<timestamp>/`) so runs are discoverable without remembering a path each time. `sandbox trace --backend` has no default yet, intentionally — the isolation-backend choice is still open.

Cross-Component Data Flow



All telemetry-producing components emit a shared telemetry record shape (command, component, stage, tokens, cost, wall-clock, model-ID, seed, timestamp) so the Evaluation Harness's cost/performance dimension can sum across stages per command. The exact schema is deferred to the data/telemetry schema pass (`08_generate_initial_data_models.md`), not designed field-by-field here.

Open Questions

- **Secure Sandbox isolation backend**: Docker/overlayfs extension vs. Firecracker vs. gVisor — research done, not yet decided (per Master Idea). `--backend` has no default until this is resolved.
- **Annotator extension work**: v1's annotator (`tracer/data.py` + `annotator/`) was not built to interpret v2-extended trace shapes (symlinks, permission-denied traces, SIGPIPE). Running v2-extended Traces through it unmodified may produce misleading annotations rather than useful ones. This is an explicit dependency for trustworthy v2-extended annotation-diff numbers, not a blocker for building the rest of the spec — where the unmodified annotator mishandles a v2-extended shape, that's itself a reportable coverage-gap finding (dimension 3), not a defect to hide.
- **Real execution-based Q1** (Tier 1, deferred): the paper's actual Q1 methodology reruns PaSh's benchmark suite, ShellCheck's 2.2K-test suite, and Shseer's 12 bug-scripts against Caruca's output — not a diff against stored ground truth. The annotation-diff comparison above is a real, useful, but distinct and lighter check. Building true per-consumer execution parity (matching the paper's 52/52-style numbers exactly) is separate, heavier infrastructure, explicitly out of scope until Tier 1.
- **Extended Traces JSON schema versioning**: strict superset of v1's `Traces` model vs. an explicitly versioned schema — needs a decision before the extended fixture fields are implemented, since v1's own tracer/annotator code reads this JSON directly.
- **Web GUI build timing**: design captured (PTY-streaming side-by-side terminal, xterm.js + websocket), but deferred behind the five priority components per the stated build order. Revisit once Baseline Instrumentation, Naive-LLM Baseline, and Evaluation Harness exist end-to-end.